

Informe de rendimiento · Yago Nexo 1.1.0

Fecha: 2026-10-04T18:43:26.208Z · Commit 7a11f60 · Máquina vm (4 vCPU, 16 GB RAM) · k6 v2.3.0 (commit/e088784614, go1.27.1, linux/amd64)

Resultado

La plataforma sostuvo la capacidad de diseño de **250 tx/s** cumpliendo los umbrales (error < 1%, p99 < 1.000 ms en REST y < 2.000 ms en integración), frente a un pico promedio de 28 tx/s de la línea base. En la prueba de estrés, el último escalón que cumplió fue **350 tx/s**.

Fases a tasa constante

Fase	Tasa objetivo (tx/s)	Tasa lograda (tx/s)	Solicitudes	Error	Operadores p50/p95/p99 (ms)	Público p50/p95/p99 (ms)	Integración p50/p95/p99 (ms)	Resultado
Nominal: pico promedio de la línea base	28	28	5.043	0.00%	5,9 / 11,5 / 19,2	8,8 / 18,1 / 28,4	17,2 / 31,7 / 77,1	Cumple
Pico esperado (ráfaga x crecimiento)	125	126	22.682	0.00%	6,5 / 20,1 / 46,8	8 / 22,4 / 47,4	17,1 / 54,7 / 112,3	Cumple
Capacidad de diseño	250	249,9	74.980	0.00%	17,8 / 127,2 / 302,2	11,7 / 76,4 / 165	49,8 / 285,5 / 644,3	Cumple

Estrés por escalones

Escalón	Tasa objetivo (tx/s)	Tasa lograda (tx/s)	Solicitudes	Error	Operadores p50/p95/p99 (ms)	Público p50/p95/p99 (ms)	Integración p50/p95/p99 (ms)	Resultado
Estrés 350 tx/s	350	348,5	31.364	0.00%	87,5 / 564,2 /	32,5 / 288,5 /	345,2 / 1.072,9 / 1.552,2	Cumple

					%	986,5	610		p99 operadores 10409. 6 ms > 1000 ms; p99 publico 1062.4 ms > 1000 ms; p99 integracion 10557. 5 ms > 2000 ms; k6 no sostuvo la tasa (5153 iteraciones no lanzadas)
Estrés 450 tx/s	450	392,8	35.349	0 . 0 6 %	1.802,7 / 6.855 / 10.409,6	61, 1 / 435 ,7 / 1.0 62, 4	3.310,2 / 7.352,7 / 10.557, 5		

Uso de recursos (CPU promedio / memoria máxima en MiB)

Fase	api m	gw- publico	mi	keycloak	postgres	rabbitmq	opensearch	concesiones-v1
Nominal: pico promedio de la línea base	2 7 % / 2. 6 7 3	16,8% / 1.354	1 7, 2 % / 8 7 7	1,3 % / 788	1,9% / 123	12,2 % / 214	2,1% / 1.483	2,5% / 55
Pico esperado (ráfaga x crecimiento)	6 0, 8 % / 2. 6 7 3	40,4% / 1.405	4 9, 5 % / 1. 0 2 8	0,3 % / 787	4,6% / 130	13,2 % / 221	6% / 1.483	8,3% / 62
Capacidad de diseño	7 6, 8 % / 2. 6 7 4	48,9% / 1.430	6 6 % / 1. 2 4 5	1,1 % / 787	7,2% / 160	19% / 237	5% / 1.484	12,8% / 69
Estrés	8	50,4% /	6	0,3	5,6%	19,5	16,5% /	12,6% / 67

350 tx/s	1, 6 %	1.430	4, 9 %	% / 784	/ 159	% / 242	1.471	
	/ 2. 6 6 3		/ 1. 6 0 3					
Estrés 450 tx/s	8 5, 1 %	57,4% / 1.428	6 5, 9 %	2,6 % / 780	5,6% / 159	16,9 % / 240	5,6% / 1.463	12,6% / 66
	/ 3. 2 2 3		/ 1. 6 5 1					

CPU en % de un núcleo (100% = 1 vCPU), según docker stats cada 3 s.

Cómo se midió

k6 genera tasa constante de llegadas (no depende de cuánto tarda cada respuesta) con la mezcla 50% gateway de operadores (REST), 30% gateway público (REST) y 20% integración (gateway → Micro Integrator con validación, llamada SOAP, transformación y publicación en RabbitMQ, con llave de idempotencia única). Cada llamada lleva un token OAuth2 de Keycloak; el token se obtiene antes de medir. La latencia es de extremo a extremo vista por el consumidor, incluido el backend de prueba.

Durante la prueba se pausó la regla del guardián de anomalías «Abuso de volumen en Concesiones», que de otro modo bloquea la carga sostenida de una sola aplicación (es lo que debe hacer); se reactivó al terminar. Laboratorio en una sola máquina compartida por unos 30 contenedores (WSO2, Keycloak, PostgreSQL, RabbitMQ, OpenSearch, observabilidad, backends de prueba y el propio generador de carga). En el laboratorio, el plano de control de WSO2 y el gateway de operadores comparten un contenedor; en producción el gateway corre en instancias dedicadas de 2 vCPU según la planilla de dimensionamiento.

Se reproduce con: `pnpm --filter @nexo/lab-bootstrap rendimiento (script tests/rendimiento/mezcla.js)`.